

MTIL — Multi-Task Incremental Learning with CLIP

Research framework for continual learning on vision-language models, built on top of OpenAI's CLIP. Supports sequential fine-tuning across multiple datasets with methods designed to prevent catastrophic forgetting.

Quick Start

```
# Fine-tune on DTD for 1000 iterations
python -m src.main \
    --method finetune \
    --train-mode whole \
    --train-dataset DTD \
    --iterations 1000 \
    --lr 1e-5 \
    --save ckpt/dtd_ft

# ZSCL with ImageNet reference
python -m src.main \
    --method ZSCL \
    --train-mode whole \
    --train-dataset DTD \
    --ref-dataset ImageNet \
    --ref-sentences conceptual_captions \
    --iterations 1000 \
    --lr 1e-5 \
    --save ckpt/dtd_zscl

# Evaluation only
python -m src.main \
    --load ckpt/dtd_ft/DTD.pth \
    --eval-datasets DTD,CIFAR100 \
    --eval-only

# Single image evaluation
python -m src.main \
    --load ckpt/dtd_ft/DTD.pth \
    --eval-single path/to/image.jpg \
    --class-names data/text_classes/imagenet_classes.txt \
    --eval-only
```

SLURM

Edit `train.sh` with your arguments, then:

```
sbatch train.sh
```

Environment Setup

```
conda env create -f environment.yml
conda activate <env_name>
# or
pip install -r requirements.txt
```

Project Structure

```
mtil/
├── clip/                                # Modified OpenAI CLIP (model, tokenizer,
loader)
├── src/
│   ├── main.py                        # Entry point
│   ├── args.py                       # All CLI arguments
│   ├── datasets/                     # Dataset implementations
│   ├── models/
│   │   └── training.py               # Main training loop (GradientTracker,
TrainingState)
│   ├── finetune.py                  # finetune / lwf / ZSCL pipelines
│   ├── icarl.py                    # iCaRL pipeline
│   ├── evaluation.py                # Zero-shot + accuracy evaluation
│   ├── helpers.py                   # WiSE-FT, L2, distillation utilities
│   ├── lora_injection.py            # Custom shared LoRA injection
│   └── modeling.py                  # ImageEncoder, ClassificationHead
├── templates/                       # Zero-shot prompt templates
├── scripts/                         # Experiment shell scripts
├── data/                           # Dataset storage root
├── ckpt/                           # Checkpoint outputs
├── logs/                           # Training logs
├── train.sh                         # SBATCH job template
├── list_clip_layers.py              # Utility: list CLIP model layers
├── datasets.md                      # Manual dataset download instructions
└── ARGUMENTS.md                    # Full argument reference
```

Training Methods

--method	Description
finetune	Standard full-model fine-tuning
lwf	Learning Without Forgetting — distills from a frozen copy of the previous model

<code>--method</code>	Description
<code>zSCL</code>	Zero-Shot Continual Learning — regularizes against a reference dataset/model to preserve zero-shot ability
<code>icarl</code>	iCaRL — exemplar memory-based incremental learning across sequential datasets

Train Modes

Controls which parts of CLIP are updated (`--train-mode`):

Mode	Trains
<code>whole</code>	Image encoder + text encoder
<code>image</code>	Image encoder only
<code>text</code>	Text encoder only
<code>image-fc</code>	Image encoder + classification head
<code>image-fc-fixed</code>	Image encoder (classification head frozen)
<code>fc</code>	Classification head only (linear probe)

CLIP Model Variants

Passed via `--model` (default: `ViT-B/16`):

Model	Architecture	Notes
<code>ViT-B/32</code>	ViT, 32×32 patch	Fastest, lowest capacity
<code>ViT-B/16</code>	ViT, 16×16 patch	Default — best speed/accuracy balance
<code>RN50</code>	ResNet-50	CNN backbone
<code>RN101</code>	ResNet-101	Larger CNN
<code>RN50×4</code>	ResNet-50 4× width	Wide CNN
<code>RN50×16</code>	ResNet-50 16× width	Extra-wide CNN

Models auto-download from OpenAI on first use.

Datasets

Auto-download (via torchvision / dataset loaders)

Name	<code>--train-dataset</code> value	Classes
Describable Textures	<code>DTD</code>	47

Name	--train-dataset value	Classes
CIFAR-10	CIFAR10	10
CIFAR-100	CIFAR100	100
Flowers102	Flowers	102
Food101	Food	101
EuroSAT	EuroSAT	10
Stanford Cars	StanfordCars	196
Oxford Pet	OxfordPet	37
SUN397	SUN397	397
Caltech101	Caltech101	102
MNIST	MNIST	10
FGVC Aircraft	Aircraft	100

Manual download required

See `datasets.md` for full instructions.

Dataset	--train-dataset / --ref-dataset
ImageNet	ImageNet
ImageNet-A	ImageNetA
ImageNet-R	ImageNetR
ImageNet-Sketch	ImageNetSketch
ImageNet-V2	ImageNetV2
ObjectNet	ObjectNet
YTBB-Robust	YTBB
ImageNet Vid-Robust	ImageNetVidRobust
Conceptual Captions	conceptual_captions (ref sentences)
FMOW (WILDS)	FMOW
IWildCam (WILDS)	IWildCam

All datasets default to `./data/` — change with `--data-location`.

Key Arguments

Full reference: `ARGUMENTS.md`

Core

Argument	Default	Description
<code>--method</code>	—	Training method (<code>finetune</code> , <code>lwf</code> , <code>ZSCL</code> , <code>icarl</code>)
<code>--train-mode</code>	—	Which parts to train (see table above)
<code>--train-dataset</code>	—	Dataset to train on
<code>--iterations</code>	—	Total training steps (mutually exclusive with <code>--epochs</code>)
<code>--epochs</code>	—	Train for N epochs instead of a fixed iteration count
<code>--lr</code>	0.001	Learning rate
<code>--batch-size</code>	8	Training batch size
<code>--save</code>	—	Directory to save checkpoints
<code>--load</code>	—	Checkpoint path to resume from

ZSCL

Argument	Default	Description
<code>--ref-dataset</code>	—	Reference dataset (e.g., <code>ImageNet</code>)
<code>--ref-sentences</code>	—	Reference text source (e.g., <code>conceptual_captions</code>)
<code>--ref-model</code>	—	Path to reference model (defaults to zero-shot CLIP)
<code>--T</code>	2.0	Distillation temperature
<code>--image-loss</code>	False	Include image-side distillation loss
<code>--text-loss</code>	False	Include text-side distillation loss

Weight Averaging / WiSE-FT

Argument	Default	Description
<code>--we</code>	False	Weight ensembling during training
<code>--we-wise</code>	False	WiSE-FT weight ensembling
<code>--we-wise-alpha</code>	0.98	WiSE interpolation coefficient
<code>--moving-avg</code>	False	Exponential moving average of weights
<code>--wise-merge</code>	False	Apply WiSE merge at end of training

OGD (Orthogonal Gradient Descent)

Argument	Default	Description
<code>--orthogonal-gradients</code>	—	Number of OGD projections to sample

Argument	Default	Description
<code>--orthogonal-gradients-path</code>	—	Path to previous task's SVD gradient basis

LoRA

Argument	Default	Description
<code>--lora</code>	False	Enable LoRA adapter training
<code>--lora-r</code>	8	Low-rank dimension
<code>--lora-alpha</code>	16	Scaling factor (effective: alpha/r)
<code>--lora-dropout</code>	0.1	Dropout on LoRA layers (peft only)
<code>--lora-target-modules</code>	<code>attn.out_proj,mlp.c_fc,mlp.c_proj</code>	Comma-separated module names
<code>--lora-shared</code>	False	Use custom shared-A LoRA instead of peft

Evaluation

Argument	Default	Description
<code>--eval-datasets</code>	—	Comma-separated datasets to evaluate
<code>--eval-interval</code>	—	Evaluate every N iterations
<code>--eval-only</code>	False	Skip training, run evaluation only
<code>--eval-single</code>	—	Path to a single image to classify

LoRA

Two backends are available:

peft library (default when `--lora` is set):

```
python -m src.main --lora --lora-r 8 --lora-alpha 16 ...
```

Custom shared LoRA (`--lora-shared`): Uses `src/models/lora_injection.py`. Shares a single A matrix across all target layers; each layer keeps its own B. No external dependency.

```
python -m src.main --lora --lora-shared --lora-r 8 --lora-alpha 16 ...
```

Inspect available layer names for targeting:

```
python list_clip_layers.py --linear-only    # only nn.Linear layers
python list_clip_layers.py --params        # all parameters with shapes
```

Continual Learning Workflow

A typical sequential multi-dataset experiment:

```
DATASETS="DTD Flowers EuroSAT MNIST StanfordCars Food CIFAR100 SUN397
Aircraft OxfordPet Caltech101"
PREV_CKPT=""

for DS in $DATASETS; do
  LOAD_ARG=""
  [ -n "$PREV_CKPT" ] && LOAD_ARG="--load $PREV_CKPT"

  python -m src.main \
    --method ZSCL \
    --train-mode whole \
    --train-dataset $DS \
    --ref-dataset ImageNet \
    --ref-sentences conceptual_captions \
    --iterations 1000 \
    --lr 1e-5 \
    --save ckpt/zscl_run \
    $LOAD_ARG

  PREV_CKPT="ckpt/zscl_run/${DS}.pth"
done
```

Pre-written scripts for common configurations are in [scripts/](#).

OGD Workflow

Collect gradients during training and project future updates to be orthogonal to past task gradients.

```
# Task 1: train and save gradient basis
python -m src.main \
  --method finetune \
  --train-dataset DTD \
  --orthogonal-gradients 10 \
  --save ckpt/ogd/task1 ...

# Task 2: project gradients away from task 1 subspace
python -m src.main \
```

```
--method finetune \  
--train-dataset Flowers \  
--orthogonal-gradients 10 \  
--orthogonal-gradients-path ckpt/ogd/task1/grad_DTD.pth \  
--save ckpt/ogd/task2 ...
```

To manually convert raw gradient files to SVD basis:

```
python convert_grad_to_basis.py --input grad_DTD.pth --output basis_DTD.pth
```

Evaluation Metrics

- **Top-1 accuracy** and **Top-5 accuracy** per dataset
- Results saved to `<save>/metrics_<dataset>.csv` (one row per evaluation checkpoint)
- Zero-shot classification uses dataset-specific prompt templates from `src/templates/`